

طراحی سؤال‌های مصاحبه‌ی فنی

راهنمای عملی برای ساختن، ارزیابی و کالیبره کردن سؤال

این سند بخشی از مخزن 1995parham-teaching/interviews است و در کنار پوشه‌های
system-design و questions، problems، code-session استفاده می‌شود.

فهرست مطالب

۱. چرا طراحی سؤال، خودش یک مسئله‌ی مهندسی است	3
۲. از شرح شغل تا سؤال	3
۲.۱. گام اول: استخراج سیگنال‌های هدف	3
۲.۲. گام دوم: نگاشت سیگنال به قالب سنجش	3
۲.۳. گام سوم: تعیین سطح سؤال	4
۳. آتاتومی یک سؤال خوب	5
۳.۱. هسته‌ی ساده	5
۳.۲. لایه‌های توسعه	5
۳.۳. صورت مسئله‌ی مکتوب	5
۳.۴. راهنمایی‌های مرحله‌بندی‌شده	5
۳.۵. سنجه	6
۴. معیارهای غربال کردن یک ایده‌ی سؤال	6
۴.۱. قابل جست‌وجو نباشد که کل امتیاز را بدهد	6
۴.۲. یک «جرقه» لازم نداشته باشد	6
۴.۳. در بودجه‌ی زمانی جا شود	7
۴.۴. به دانش دامنه‌ی خاص وابسته نباشد	7
۴.۵. چند راه‌حل معتبر داشته باشد	7
۵. ضدالگوها	7
۶. کالیبراسیون: سؤال را قبل از آدم‌ها تست کنید	8
۷. هماهنگی بین سؤال‌ها	8
۸. نوشتن سنجه بدون گرفتار شدن در تعارف	9
۹. چند نمونه‌ی بازطراحی‌شده	9
۹.۱. از سؤال بد به سؤال خوب	9
۹.۲. از سؤال باز به سؤال قیددار	10
۱۰. سیاهه‌ی بازبینی نهایی	10

۱. چرا طراحی سؤال، خودش یک مسئله‌ی مهندسی است

مصاحبه‌ی فنی یک آزمایش اندازه‌گیری است: می‌خواهیم از روی یک نمونه‌ی یک‌ساعته پیش‌بینی کنیم که یک نفر در طول ماه‌ها و سال‌ها چطور کار خواهد کرد. مثل هر ابزار اندازه‌گیری دیگری، کیفیت نتیجه بیشتر از آن‌که به مهارت مصاحبه‌گر وابسته باشد، به کیفیت خود ابزار وابسته است. سؤال بد را با مصاحبه‌گر خوب هم نمی‌شود نجات داد.

سه ویژگی، یک سؤال را از یک «چیستان» به یک ابزار سنجش تبدیل می‌کند:

اعتبار (Validity) سؤال دقیقاً همان چیزی را می‌سنجد که ادعا می‌کند. سؤالی که در ظاهر درباره‌ی طراحی سیستم است ولی عملاً بلد بودن یک اصطلاح خاص را می‌سنجد، اعتبار ندارد.

پایایی (Reliability) اگر دو مصاحبه‌گر مختلف همان سؤال را از همان فرد بپرسند، به نتیجه‌ی نزدیک به هم برسند. پایایی از سنج (Rubric) می‌آید، نه از حس شخصی.

عدالت (Fairness) سؤال نباید به کسی که پیش‌زمینه، زبان مادری یا مسیر شغلی متفاوتی داشته امتیاز منفی بدهد، مگر آن‌که همان تفاوت واقعاً برای شغل موضوعیت داشته باشد.

اصل راهنما

هر سؤال باید به این جمله پاسخ بدهد: «اگر کسی این سؤال را خوب جواب بدهد، دقیقاً چه چیزی درباره‌ی عملکرد او در این نقش فهمیده‌ام؟» اگر جواب این سؤال مبهم است، سؤال هنوز آماده نیست.

۲. از شرح شغل تا سؤال

طراحی سؤال از خود سؤال شروع نمی‌شود؛ از کاری شروع می‌شود که قرار است فرد انجام بدهد.

۲.۱. گام اول: استخراج سیگنال‌های هدف

فهرستی از کارهای واقعی روزمره‌ی آن نقش بنویسید و از دل آن‌ها، چند سیگنال بیرون بکشید. سیگنال یعنی یک توانایی مشخص و قابل مشاهده. برای مثال، برای یک مهندس بک‌اند:

- ترجمه‌ی یک نیازمندی مبهم به یک مدل داده و یک قرارداد
- انتخاب ساختمان داده‌ی مناسب و توجیه هزینه‌ی زمانی و حافظه‌ای آن
- نوشتن کد قابل تست و پیش‌بینی حالت‌های مرزی
- رفتار درست در برابر خطا، هم‌زمانی و شکست‌های شبکه
- توضیح دادن تصمیم‌های فنی برای کسی که با مسئله آشنا نیست

۲.۲. گام دوم: نگاشت سیگنال به قالب سنجش

هر سیگنال با یک قالب بهتر سنجیده می‌شود. تلاش برای سنجیدن همه چیز با یک قالب، شایع‌ترین اشتباه در طراحی فرایند مصاحبه است.

قالب	چه چیزی را خوب می‌سنجد	چه چیزی را نمی‌سنجد
پرسش شفاهی	وسعت دانش، دقت مفاهیم، تجربه‌ی واقعی	توانایی ساختن چیزی
مسئله‌ی الگوریتمی	استدلال، تحلیل پیچیدگی، حالت‌های مرزی	طراحی، خوانایی، کار با کد موجود
جلسه‌ی کدنویسی	ساختاردهی، تست‌پذیری، کیفیت کد	عمق نظری
طراحی سیستم	مصالحه‌ها، مقیاس، شکست‌ها	مهارت عملی کدنویسی
بازبینی کد	دقت، نگاه انتقادی، ارتباط	توانایی تولید از صفر

این دقیقاً همان چیزی است که ساختار این مخزن روی آن بنا شده است: پوشه‌های `questions`، `problems`، `code-session` و `system-design` چهار قالب متفاوت‌اند، نه چهار سطح سختی.

۲.۳. گام سوم: تعیین سطح سؤال

بعد از انتخاب قالب، باید مشخص کنید سؤال روی چه **باری** کالبره شده است. سطح، معیار باهوشی سؤال نیست؛ معیار این است که سؤال با چه انتظاری از نقش سنجیده می‌شود. پرسیدن سؤالی بالاتر از سطح داوطلب، سیگنال قوی‌تری نمی‌دهد – فقط سیگنال پرنویزتری می‌دهد: می‌فهمید که کسی گیر کرد، ولی نمی‌فهمید در کاری که واقعاً برایش استخدام می‌شود هم گیر می‌کند یا نه.

در این مخزن، سؤال‌های پوشه‌ی `problems` به سه سطح تقسیم شده‌اند:

سطح ۱ – فرانت‌اند و SRE مبانی. آرایه، رشته، جدول درهم‌ساز و شبیه‌سازی مستقیم همان چیزی که در صورت مسئله توصیف شده. هیچ الگوریتمی لازم نیست **کشف** شود؛ داوطلب باید یک مشخصات روشن را به کد درست ترجمه کند و حالت‌های مرزی را ببیند.

بودجه: ۲۰ تا ۲۵ دقیقه. **دنبال چه هستید:** تجزیه‌ی درست مسئله، حالت‌های مرزی (ورودی خالی، تک‌عضوی، تساوی)، نام‌گذاری خوانا و توانایی بیان مرتبه‌ی زمانی. **امتیاز منفی ندهید** بابت نرسیدن به راه‌حل بهینه‌تر؛ سیگنال این سطح آن نیست.

سطح ۲ – مهندسان بک‌اند انتخاب ساختمان داده‌ی درست و توجیه آن. پشته، درخت پیشوندی، هرم، گراف روی شبکه، پنجره‌ی لغزان و بازگشت با حالت. یک راه‌حل ساده وجود دارد و یک راه‌حل بهتر، و فاصله‌ی بین این دو، بخش جالب گفت‌وگوست.

بودجه: ۳۰ تا ۴۰ دقیقه. **دنبال چه هستید:** شروع از راه‌حل ساده و بهبود آگاهانه‌ی آن، تحلیل درست پیچیدگی هر دو، و انتخاب ساختمان داده پیش از نوشتن کد. **جواب درست بدون بحث پیچیدگی را نپذیرید؛** در این سطح استدلال از خود کد ارزشمندتر است.

سطح ۳ – سنورها راه‌حل بهینه به یک ناوردا، یک قید فشرده‌ی حافظه، یا تصمیمی که جواب یکتا ندارد وابسته است. بسیاری از این سؤال‌ها از داوطلب می‌خواهند چیزی را **طراحی** کند (یک واسط، یک ساختمان داده با ناوردا، نگه‌داشته‌شده) نه فقط یک جواب حساب کند.

بودجه: ۴۵ تا ۶۰ دقیقه، و حتماً با لایه‌ها – به چشم قبول/رد به آن نگاه نکنید. **دنبال چه هستید:** تشخیص همان قیدی که راه‌حل ساده را غیرقابل قبول می‌کند، پیشنهاد بیش از یک راه‌حل و انتخاب بلند بین آن‌ها، و زیر سؤال بردن فرض‌های صورت مسئله. **سطح ۳ را تنها سؤال کدنویسی یک فرایند نکنید؛** آن را با یک سؤال سطح ۲ جفت کنید تا از داوطلبی که آن را باز نکرد هم سیگنال قابل استفاده بگیرید.

سطح، جای لایه‌ها را نمی‌گیرد

یک سؤال سطح ۲ که خوب لایه‌بندی شده باشد، از یک داوطلب سنیور هم سیگنال می‌گیرد، چون لایه‌های بالاتر او را به مرز توانایی‌اش می‌برد. برعکسش درست نیست: سؤال سطح ۳ برای داوطلب سطح ۱ فقط یک صفحه‌ی خالی تولید می‌کند. وقتی شک دارید، سؤال پایین‌تر با لایه‌های بیشتر را انتخاب کنید.

۳. آناتومی یک سؤال خوب

یک سؤال آماده، چند جزء دارد. اگر هرکدام از این‌ها را ننویسید، در جلسه مجبور به بداهه‌گویی می‌شوید و پایایی از بین می‌رود.

۳.۱. هسته‌ی ساده

سؤال باید با یک نسخه‌ی ساده شروع شود که هر داوطلب واجد شرایطی بتواند در ۱۰ دقیقه از پشش بریباید. هدف این نیست که کسی رد شود؛ هدف این است که یک نقطه‌ی شروع مشترک بسازید تا بتوانید از آن به بالا حرکت کنید. سؤال‌ی که ۴۰ درصد داوطلبان حتی نمی‌توانند شروعش کنند، هیچ اطلاعاتی درباره‌ی آن ۴۰ درصد به شما نمی‌دهد.

۳.۲. لایه‌های توسعه

بعد از هسته، باید چند «لایه» از پیش آماده داشته باشید تا سؤال را عمیق‌تر کنید. این لایه‌ها همان چیزی هستند که یک سؤال را از سنجش دودویی (بلد است / بلد نیست) به سنجش پیوسته (تا کجا بلد است) تبدیل می‌کنند. برای مثال، مسئله‌ی «کوتاه‌کننده‌ی نشانی» را در نظر بگیرید:

1. **هسته:** یک نگاشت از نشانی بلند به کوتاه بساز.
2. **لایه:** شناسه چطور تولید می‌شود؟ برخورد را چطور مدیریت می‌کنی؟
3. **لایه:** حالا روزی صد میلیون درخواست خواندن داریم. چه چیزی اول می‌شکند؟
4. **لایه:** نشانی‌ها باید منقضی شوند. این چطور روی طراحی ذخیره‌سازی اثر می‌گذارد؟
5. **لایه:** یک دیتاست از دسترس خارج می‌شود. چه چیزی را قربانی می‌کنی؟

۳.۳. صورت مسئله‌ی مکتوب

صورت مسئله را دقیقاً همان‌طور که می‌خواهید بخوانید بنویسید و برای همه یکسان نگه دارید. ورودی، خروجی، محدودیت‌ها و حداقل دو مثال باید نوشته شده باشد. اگر سؤال را هر بار با کلمات متفاوتی بپرسید، عملاً هر بار سؤال دیگری پرسیده‌اید.

۳.۴. راهنمایی‌های مرحله‌بندی‌شده

از قبل بنویسید که اگر داوطلب گیر کرد، چه راهنمایی‌ای و با چه ترتیبی می‌دهید و هر راهنمایی چقدر از امتیاز را جبران می‌کند. بدون این، حجم راهنمایی به میزان همدلی شما در آن روز بستگی پیدا می‌کند و مقایسه‌ی داوطلبان بی‌معنا می‌شود.

اندازه‌ی راهنمایی

ترتیب درست راهنمایی، از کم‌ترین دخالت به بیش‌ترین:

1. سکوت کردن و فضا دادن.
 2. پرسیدن «الان داری به چه چیزی فکر می‌کنی؟»
 3. اشاره به یک مثال نقض.
 4. اشاره به دسته‌ی راه‌حل («اگر بخواهی چیزی را به یاد بسپاری چه؟»).
 5. دادن ایده‌ی اصلی.
- هر پله‌ای که استفاده می‌کنید را ثبت کنید.

۳.۵. سنجه

سنجه یعنی سطح‌های عملکرد را از پیش و به زبان رفتار قابل مشاهده توصیف کنید، نه با صفت‌هایی مثل «خوب بود» یا «قوی بود». نمونه‌ی یک سنجه برای یک مسئله‌ی الگوریتمی:

سطح	رفتار قابل مشاهده
پایین‌تر از حد	بدون راهنمایی به راه‌حل درست نمی‌رسد؛ حالت‌های مرزی را حتی بعد از اشاره نمی‌بیند.
حد انتظار	راه‌حل درست را با یکی دو راهنمایی می‌سازد؛ پیچیدگی را درست می‌گوید؛ با اشاره، حالت‌های مرزی را می‌بیند.
بالا تر از حد	مستقل به راه‌حل بهینه می‌رسد؛ خودش مصالحه‌ها را مطرح می‌کند؛ پیش از کدنویسی مثال نقض می‌سازد.
برجسته	مسئله را تعمیم می‌دهد، فرض‌های ضمنی صورت مسئله را زیر سؤال می‌برد و راه‌حل را نقد می‌کند.

۴. معیارهای غربال کردن یک ایده‌ی سؤال

پیش از آن‌که یک ایده را وارد بانک سؤال کنید، از این فیلترها ردش کنید.

۴.۱. قابل جست‌وجو نباشد که کل امتیاز را بدهد

اگر جواب کامل سؤال با یک جست‌وجو پیدا می‌شود، شما حافظه را می‌سنجید نه تفکر را. راه‌حل، کنار گذاشتن مسئله‌های شناخته‌شده نیست؛ راه‌حل این است که امتیاز اصلی را روی لایه‌هایی بگذارید که در آن جواب آماده وجود ندارد: انتخاب بین دو راه‌حل، حالت‌های مرزی دامنه‌ی خاص شما، یا تغییر دادن صورت مسئله در وسط جلسه.

۴.۲. یک «جرقه» لازم نداشته باشد

سؤالی که یا آن ترفند خاص به ذهنت می‌رسد یا هیچ، یک سکه‌ی شیر یا خط است. سؤال خوب مسیر تدریجی دارد: راه‌حل ساده‌ی کند، بعد بهبود، بعد بهینه. اگر بین نسخه‌ی ساده و نسخه‌ی بهینه‌ی سؤال شما هیچ پله‌ی میانی وجود ندارد، سؤال را عوض کنید.

۴.۳. در بودجه‌ی زمانی جا شود

برای هر سؤال، زمان واقعی را با احتساب توضیح صورت مسئله، استرس داوطلب و پرسش و پاسخ تخمین بزنید – نه با معیار زمانی که خودتان روی کاغذ حلش می‌کنید. یک قاعده‌ی سرانگشتی: زمان حل خودتان را در سه ضرب کنید. اگر جا نشد، سؤال را کوچک کنید، نه جلسه را فشرده.

۴.۴. به دانش دامنه‌ی خاص وابسته نباشد

مگر آن‌که آن دانش دقیقاً همان چیزی باشد که استخدام می‌کنید. سؤالی که برای حل شدن به شناخت قواعد مالیاتی، قوانین شطرنج یا یک اصطلاح داخلی شرکت شما نیاز دارد، دارد آشنایی را به جای توانایی می‌سنجد.

۴.۵. چند راه حل معتبر داشته باشد

بهترین سؤال‌ها آن‌هایی هستند که بیش از یک جواب درست دارند، چون آن وقت می‌توانید درباره‌ی انتخاب گفت‌وگو کنید و انتخاب، جایی است که تجربه خودش را نشان می‌دهد.

۵. ضدالگوها

سؤال‌هایی که سیگنال تولید نمی‌کنند

اگر هر کدام از این‌ها را در بانک سؤال خود دارید، بازنگری کنید.

چیستان مسئله‌هایی که به یک بینش غیرقابل حدس متکی‌اند. رابطه‌ی این‌ها با عملکرد شغلی عملاً هیچ است و فقط اضطراب تولید می‌کنند.

بازی حدس زدن سؤالی که یک جواب «درست» در ذهن مصاحبه‌گر دارد و داوطلب باید آن را حدس بزند. نشانه‌اش این است که خودتان را در حال گفتن «نه، منظورم چیز دیگری بود» می‌بینید.

سؤال دانش‌نامه‌ای «سطح ایزولاسیون پیش‌فرض این پایگاه داده چیست؟» این یک واقعیت است که در سی ثانیه پیدا می‌شود. به جای پرسیدن: «این دو تراکنش هم‌زمان اجرا می‌شوند؛ چه اتفاقی ممکن است بیفتد و چطور جلوییش را می‌گیریم؟»

کار واقعی ناشناخته دادن یک باگ واقعی حل‌نشده از محصول خودتان. ناعادلانه است، چون زمینه‌ی ذهنی شما را ندارد، و مرز اخلاقی کار بدون دستمزد را رد می‌کند.

سؤال بیش از حد باز «یک سیستم توزیع‌شده طراحی کن.» بدون قید و نیازمندی، داوطلب نمی‌داند چه چیزی از او خواسته شده و شما هم نمی‌دانید بر چه اساسی نمره بدهید.

تله‌ی زبان سؤالی که روی یک رفتار عجیب و کم‌کاربرد یک زبان تکیه دارد. تفاوت بین «چرا این کد رفتار غیرمنتظره‌ای دارد؟» و «آیا این جزئیات را حفظ کرده‌ای؟» را جدی بگیرید.

چند سؤال در یک سؤال طراحی‌ای که در آن اگر داوطلب بخش اول را نتواند، هیچ داده‌ای از بخش‌های بعد به دست نمی‌آورد. بخش‌ها را مستقل کنید یا جواب بخش اول را در صورت نیاز خودتان بدهید و ادامه دهید.

۶. کالیراسیون: سؤال را قبل از آدم‌ها تست کنید

یک سؤال تازه طراحی شده، یک فرضیه‌ی تست نشده است. پیش از استفاده‌ی رسمی:

1. خودتان حلش کنید، اما با شرایط داوطلب: بدون آماده‌سازی قبلی و با محدودیت زمانی.
2. روی دو یا سه همکار اجرا کنید که سطح‌های متفاوتی دارند. اگر همکار ارشد شما هم گیر می‌کند، سؤال بیش از حد سخت است یا صورت مسئله مبهم است.
3. صورت مسئله را از روی سؤالات همکاران بازنویسی کنید. هر جایی که کسی مجبور شد توضیح بخواهد، یک ابهام در متن است.
4. سنجه را با داده‌ی واقعی تنظیم کنید. بعد از پنج یا شش مصاحبه، توزیع نمرات را نگاه کنید. اگر همه در یک سطح افتاده‌اند، سؤال قدرت تفکیک ندارد.

نشانه‌های سؤالی که باید بازنشسته شود

توزیع نمرات دوقطبی شده است (همه یا عالی یا افتضاح)؛ داوطلبان راه‌حل بهینه را بی‌درنگ و بدون استدلال می‌نویسند (نشانه‌ی نشت سؤال)؛ یا شما در حال توضیح دادن بیش از حد صورت مسئله هستید.

سؤال‌ها را مثل کد نسخه‌بندی کنید: هر تغییر در صورت مسئله یا سنجه را ثبت کنید، چون نمرات قبل و بعد از تغییر با هم قابل مقایسه نیستند.

۷. هماهنگی بین سؤال‌ها

سؤال‌ها را جدا جدا طراحی نکنید؛ مجموعه را طراحی کنید. برای یک فرایند کامل، یک ماتریس پوشش بسازید: در سطرها سیگنال‌ها و در ستون‌ها جلسه‌ها. هر سیگنال مهم باید دست‌کم در دو جلسه پوشش داده شود (تا خطای یک مصاحبه‌گر جبران شود) و هیچ سیگنالی نباید سه بار تکرار شود.

سیگنال	شفاهی	کدنویسی	طراحی
استدلال الگوریتمی	—	✓	—
کیفیت و ساختار کد	—	✓	—
مصالحه‌های معماری	✓	—	✓
رفتار در برابر خطا	✓	✓	✓
ارتباط و توضیح‌دهی	✓	✓	✓

ستون آخر این ماتریس به شما نشان می‌دهد کجا سؤال کم دارید — و مهم‌تر، کجا سه سؤال دارید که همگی یک چیز را می‌سنجند.

۸. نوشتن سنجه بدون گرفتار شدن در تعارف

بزرگ‌ترین دشمن پایایی، صفت‌های مبهم است. این جمله‌ها را در گزارش‌ها ممنوع کنید: «حس خوبی داشتم»، «باهوش به نظر می‌رسید»، «به فرهنگ ما می‌خورد». هر کدام از این‌ها یک قضاوت بدون شاهد است.

به جایش، الگوی **شاهد** ← **تفسیر** ← **نمره** را اجباری کنید:

شاهد بدون راهنمایی، پیش از نوشتن کد پرسید که آرایه‌ی ورودی می‌تواند خالی باشد یا نه، و بعد یک تست برای آن حالت نوشت.

تفسیر عادت به فکر کردن به حالت‌های مرزی پیش از پیاده‌سازی دارد.

نمره بالاتر از حد انتظار در «تست‌پذیری و حالت‌های مرزی».

اگر نمی‌توانید شاهد بنویسید، یعنی نمره‌تان مبنایی ندارد. این تنها قاعده‌ای است که بیشترین اثر را روی کیفیت تصمیم‌استخدام دارد.

۹. چند نمونه‌ی بازطراحی‌شده

۹.۱. از سؤال بد به سؤال خوب

نسخه‌ی ضعیف «تفاوت array و slice در Go چیست؟»
یک سؤال حافظه‌ای با جواب دودویی. یا حفظ کرده یا نکرده.

نسخه‌ی بهتر این قطعه کد را نشان بده و بپرس خروجی چیست و چرا:

```
s := []int{1, 2, 3, 4}
t := s[:2]
t = append(t, 99)
fmt.Println(s)
```

مدل ذهنی فرد از حافظه‌ی زیرین را می‌سنجد، مسیر تدریجی دارد (اول خروجی، بعد چرایی، بعد «چطور از این باگ جلوگیری می‌کنی؟») و برای کسی که واقعاً با Go کار کرده جواب دادنش طبیعی است.

۹.۲. از سؤال باز به سؤال قیددار

نسخه‌ی ضعیف «یک سیستم رأی‌گیری طراحی کن.»
بدون قید، معلوم نیست چه چیزی سنجیده می‌شود.

نسخه‌ی بهتر «یک سیستم رأی‌گیری با یک مرکز و چند شعبه داریم. سه سناریو را جداگانه بررسی کن: (۱) شبکه بی‌عیب است، (۲) شبکه به صورت تصادفی قطع می‌شود، (۳) کاربران ممکن است چند بار رأی بدهند. در هر سناریو چه چیزی را فدا می‌کنی؟»
هدف سنجش (درک قضیه‌ی CAP و مصالحه‌ها) صریح است، لایه‌ها از پیش مشخص‌اند، و در هر لایه یک تصمیم واقعی وجود دارد که می‌شود درباره‌ی آن گفت‌وگو کرد.

۱۰. سیاهه‌ی بازبینی نهایی

پیش از افزودن یک سؤال به بانک سؤال، همه‌ی این موارد باید علامت بخورد:

- ☐ سیگنال یا سیگنال‌هایی که می‌سنجد صریحاً نوشته شده است.
- ☐ سطح آن (۱، ۲ یا ۳) مشخص شده و بودجه‌ی زمانی متناسب با همان سطح است.
- ☐ یک هسته‌ی ساده دارد که تقریباً همه‌ی داوطلبان واجد شرایط از پشش برمی‌آیند.
- ☐ دست‌کم سه لایه‌ی توسعه با ترتیب مشخص دارد.
- ☐ صورت مسئله‌ی مکتوب با ورودی، خروجی، محدودیت و دو مثال دارد.
- ☐ راهنمایی‌های مرحله‌بندی‌شده و اثرشان بر نمره نوشته شده است.
- ☐ سنجه با توصیف رفتاری برای هر سطح دارد.
- ☐ با یک جست‌وجوی ساده کامل حل نمی‌شود.
- ☐ به یک جرقه‌ی ناگهانی وابسته نیست؛ مسیر پله‌پله دارد.
- ☐ به دانش دامنه‌ای خارج از نیاز شغل وابسته نیست.
- ☐ در بودجه‌ی زمانی جلسه، با احتساب توضیح و پرسش‌وپاسخ، جا می‌شود.
- ☐ دست‌کم روی دو همکار با سطح متفاوت آزمایش شده است.
- ☐ در ماتریس پوشش، سیگنالی را پوشش می‌دهد که سه بار تکرار نشده است.
- ☐ یک نسخه و تاریخ دارد تا تغییراتش قابل ردیابی باشد.

این سند بخشی از مخزن [interviews](#) است. برای ساختن یا اصلاح سؤال‌ها، پوشه‌های `questions`، `problems`، `code` و `session` و `system-design` را ببینید.